

ANÁLISE ESTÁTICA DE CÓDIGO FONTE: SONARQUBE

O Sonar (ou SonarQube) é uma agregador de métricas que pode ser usado para medir a qualidade do código do seu sistema.

Obs.: Vários concursos têm cobrado a ferramenta Sonar. Basicamente, ela realiza aquilo que é tratado nas aulas de análise estática de código, nas aulas de qualidade de software. Para aprofundar na teoria por trás da ferramenta, recomenda-se assistir essas aulas.

Análise estática de código consiste, basicamente, em um tipo de teste que não exige a execução do software, sendo diferente dos testes que, na maioria das vezes, envolvem rodar o software.

Houve um concurso em que, na prova discursiva, foi solicitado que se falasse sobre SAST e DAST. Esses termos são siglas. O SAST corresponde à análise estática e o DAST à análise dinâmica. Não há complexidade, trata-se apenas disso.

Alguns alunos relataram, durante a redação, que não sabiam o que escrever. No entanto, bastava lembrar do conceito de análise estática de código. A banca utiliza esses termos em inglês para gerar confusão. Portanto, SAST é uma ferramenta de análise estática. Esse termo não é amplamente utilizado no mercado, mas a banca insiste em repeti-lo. Na prática, trata-se apenas de análise estática em inglês.

Ele é feito em Java e roda métricas para diversas linguagens (como Java, C++, JavaScript e CSS); a maioria delas de forma gratuita, e algumas pagas. O seu site oficial é <http://www.sonarqube.org/>. Atualmente, na versão 25.

Obs.: O Sonar possui uma versão *Community* e, obviamente, uma versão mantida por uma empresa, a própria empresa Sonar. No site, essa empresa oferece uma série de plugins. A versão *Community* permite realizar diversas análises, mas, para recursos mais específicos, é possível adquirir licenças.

Por exemplo, no próprio órgão, o TRS, foi vivenciada essa situação. O Sonar é utilizado para várias linguagens de programação do mercado, como PHP. No entanto, também havia o Oracle Forms, que possui um plugin específico.



SOLUÇÕES

Atualmente, o Sonar possui uma versão em *cloud*, podendo ser executado na nuvem. É possível optar por essa versão ou baixar o servidor, que é o procedimento normalmente adotado, subindo uma instância desse servidor do Sonar. Também há quem realize o acoplamento do Sonar com a própria IDE, permitindo que, no momento da programação, a análise estática já seja realizada.

Análise estática consiste em observar o código-fonte a partir de alguns parâmetros, com o objetivo de identificar problemas nesse código, sem a necessidade de executá-lo. O Sonar possui uma gama extensa de problemas que consegue identificar, incluindo o chamado *code smell*.

Code smell significa, literalmente, “cheiro de código”, ou seja, um código que apresenta má qualidade. Isso ocorre, por exemplo, quando há ausência de uma indentação adequada. Atualmente, a maioria das IDEs já realiza a indentação automaticamente. Outra situação é o uso de nomes de variáveis sem significado, como A, B ou C, o que compromete a manutenibilidade do código. A ferramenta analisa todos esses aspectos.



SONARQUBE CLOUD

**For cloud-powered
DevOps**

Integrate SonarQube Cloud with your cloud DevOps platform to ensure code quality and code security, maintain high standards, and protect your code from vulnerabilities.

[Try SonarQube Cloud >](#)



SONARQUBE SERVER

**Self-managed for
ultimate control**

Ensure code quality and code security meet high standards early, before reaching production. Integrates into your enterprise DevOps environment to easily find and fix coding issues within your current workflow.

[Try SonarQube Server >](#)



SONARQUBE FOR IDE

**Within the developer
flow**

Up your coding game and find issues early. SonarQube for IDE takes linting to another level, empowering you to prevent issues as you code, no matter what languages or tools you use.

[Install SonarQube for IDE >](#)

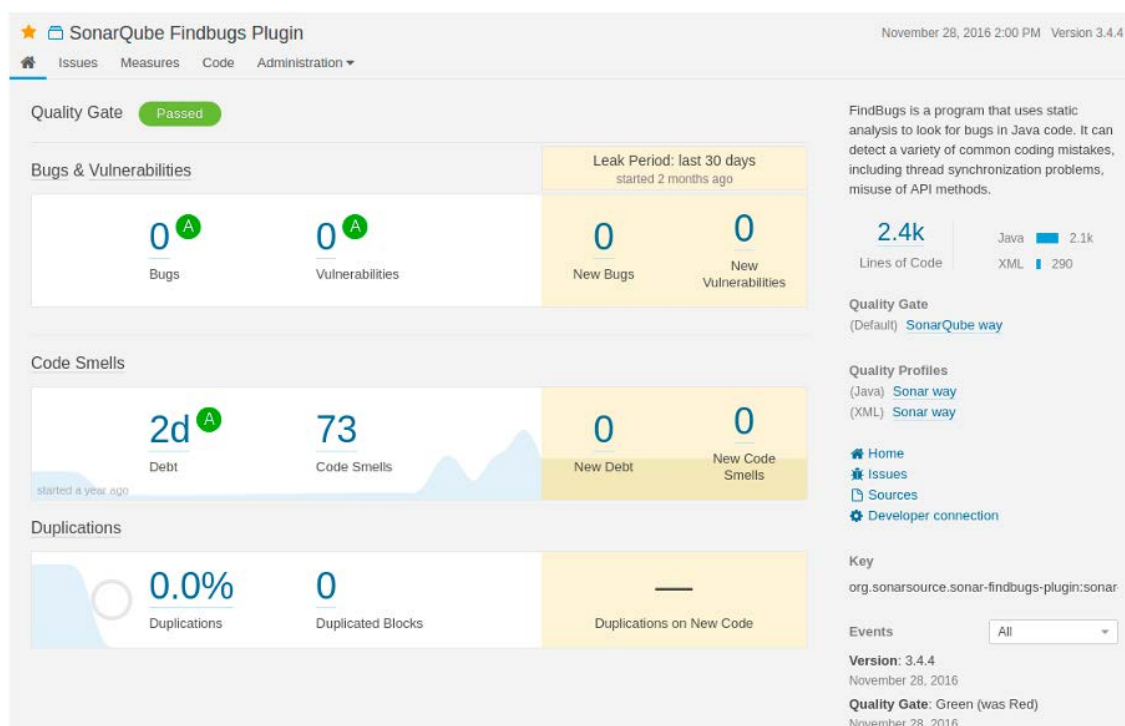
COMO USÁ-LO?

O uso do Sonar consiste em duas partes:

1. Gerar métricas por meio do plugin do Maven (`mvn sonar:sonar`);
2. Consultar as métricas geradas no site do Sonar. Para gerar as métricas em projetos Java, usamos o plugin do Maven; para as linguagens restantes, é utilizado o SonarQube Runner.

Basicamente, é possível realizar o seguinte procedimento: utilizar o servidor, apontando-o para o versionador, onde o código está implantado, por exemplo, um Git ou GitLab. O Sonar é executado sobre esse repositório. Isso pode ser feito por meio de plugins, como o do Maven.

No servidor, é possível consultar as métricas que foram geradas. Esse plugin realiza a análise diretamente sobre o código local. A versão apresentada é uma versão antiga do Sonar.



Na imagem, é possível observar alguns dos itens que são analisados, como a quantidade de *bugs*, a quantidade de vulnerabilidades, débito técnico, *code smells* e duplicações.

O débito técnico ocorre quando o código é desenvolvido sem, por exemplo, a realização de testes para determinada parte, ficando pendências a serem resolvidas posteriormente. É necessário controlar esse débito para que não aumente excessivamente. O débito técnico ocorre, principalmente, quando se utilizam métodos ágeis, onde há entregas incrementais do software. A cada incremento, é necessário otimizar o código, caso contrário, o débito técnico se acumula com soluções provisórias que, muitas vezes, não são mais revisadas.

As duplicações também representam um problema. Quando há, por exemplo, uma função que, em vez de ser reutilizada por meio de herança, polimorfismo ou boas práticas de desenvolvimento, é simplesmente copiada para outro local, qualquer *bug* encontrado exigirá correção em todos os pontos onde a função foi duplicada. Isso gera desorganização e dificuldade de manutenção. O Sonar realiza a análise desse tipo de problema, entre outras métricas.

A imagem ampliada mostra esse painel do Sonar, que apresenta os problemas identificados, as vulnerabilidades e as questões de segurança. A ferramenta realiza análise de segurança, incluindo itens como *injection* e outros problemas que estão listados no Top 10 da OWASP. Isso é especialmente relevante para áreas como a perícia da Polícia Federal, que estuda segurança da informação.

Essas vulnerabilidades indicam que o código não foi desenvolvido seguindo princípios de desenvolvimento seguro. Não há necessidade de se aprofundar nos detalhes de

implementação, pois as questões cobradas exigem o entendimento geral da ferramenta, não sendo apresentadas telas específicas para interpretação.

SONARQUBE

- O SonarQube pode registrar o histórico de métricas e fornecer gráficos de evolução.

Obs.: Executa-se de tempo em tempo. Isso é utilizado em vários lugares nos quais foi trabalhado e também em projetos de pesquisa. Realiza-se uma entrega, o fornecedor faz uma entrega, executa-se o Sonar e verifica-se a situação. O Painel exibe dois dias de débito técnico, setenta e três *code smells*, duplicação passou zerado, bugs e vulnerabilidades passaram zerados. Esse cenário está muito bom. Há dois mil e quatrocentas linhas, sendo duas mil e cem em Java e duzentas e noventa em XML, e o cenário apresentado está satisfatório. O Quality Gate foi aprovado.

O conceito de Quality Gate no Sonar normalmente possui níveis, funcionando como um índice olímpico. Para ser aprovado, é necessário alcançar o índice estabelecido. O Quality Gate define um limite aceitável em percentual. Caso esse nível não seja atingido, são apontados os pontos que precisam ser ajustados.

- O SonarQube fornece análise e integração totalmente automatizadas com Maven, Ant, Gradle, MSBuild e ferramentas de integração contínua (Atlassian Bamboo, Jenkins, Hudson, etc.).
- Plataforma de código aberto desenvolvida pela SonarSource para inspeção contínua da qualidade do código para executar revisões automáticas com análise estática do código para detectar bugs, odores (smells) de código e vulnerabilidades de segurança em mais de 20 linguagens de programação.
- SonarQube oferece relatórios sobre código duplicado, padrões de codificação, testes unitários, cobertura de código, a complexidade do código, comentários, erros e vulnerabilidades de segurança.

Obs.: Ou seja, aquilo que é aprendido em programação, sobre boas práticas, estudado na engenharia de software e na construção de software, o Sonar realiza essa verificação. Não é necessário rodar a aplicação, pois trata-se de uma análise estática. O Sonar realiza essa análise. No passado, antes da existência do Sonar, essa verificação era feita por meio de check list. Realizava-se a análise manual do código, identificando se determinada variável estava inadequada ou se havia algum problema. Atualmente, a ferramenta identifica o padrão e realiza essa verificação.



10m

CARACTERÍSTICAS

- Inclui suporte para as linguagens de programação Java (incluindo Android), C #, PHP, JavaScript, TypeScript, C/C ++, Ruby, Kotlin, Go, COBOL, PL/SQL, PL/I, ABAP, VB.NET, VB6, Python, RPG, Flex, Objective-C, Swift, CSS, HTML e XML.
- Alguns destes estão disponíveis apenas através de uma licença comercial.
- Está disponível gratuitamente sob a **GNU Lesser General Public License**. Também existe uma versão corporativa para licenciamento pago, além de uma edição de data center que suporta alta disponibilidade.

PLUGINS

- Se integra aos ambientes de desenvolvimento Eclipse, Visual Studio e IntelliJ IDEA através do SonarLint plug-ins e também se integra a ferramentas externas como LDAP, Active Directory, GitHub e outras.
- O SonarQube é expansível com o uso de plug-ins.

The image shows a promotional banner for SonarQube with the text "Your teammate for Code Quality and Security". Below this, it states "SonarQube empowers all developers to write cleaner and safer code. Join an Open Community of more than 120k users." and includes a "Download" button. A note mentions "SonarQube 7.9 LTS released on July 1st! See new features".

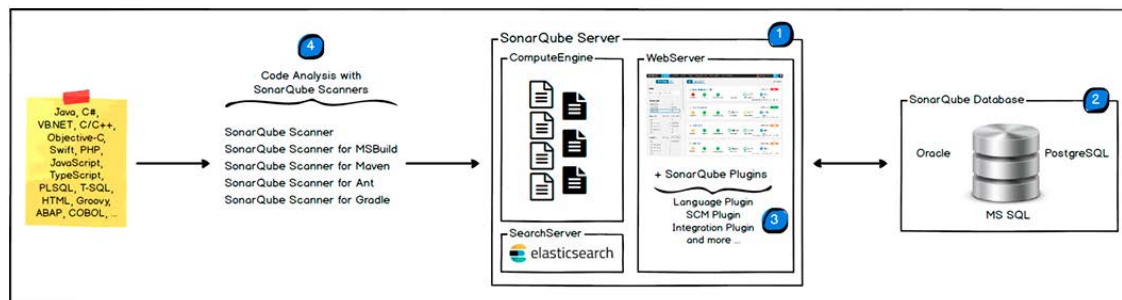
Below the banner, there is a code editor snippet showing Java code with SonarLint annotations. A red box highlights an issue: "A 'NullPointerException' could be thrown; 'providedClass' is nullable here." with a severity of "Major".

To the right of the code editor, there is a dashboard showing quality metrics:

Metric	Current Value	Grade	New code (Since last release)	Grade
Reliability				
Bugs	2	B	1	B
Security				
Security Vulnerabilities	0	A	0	A
Security Hotspots	39	-	0	-
Maintainability				
Technical Debt	6 days	C	0	A
Code Smells	319	-	0	-

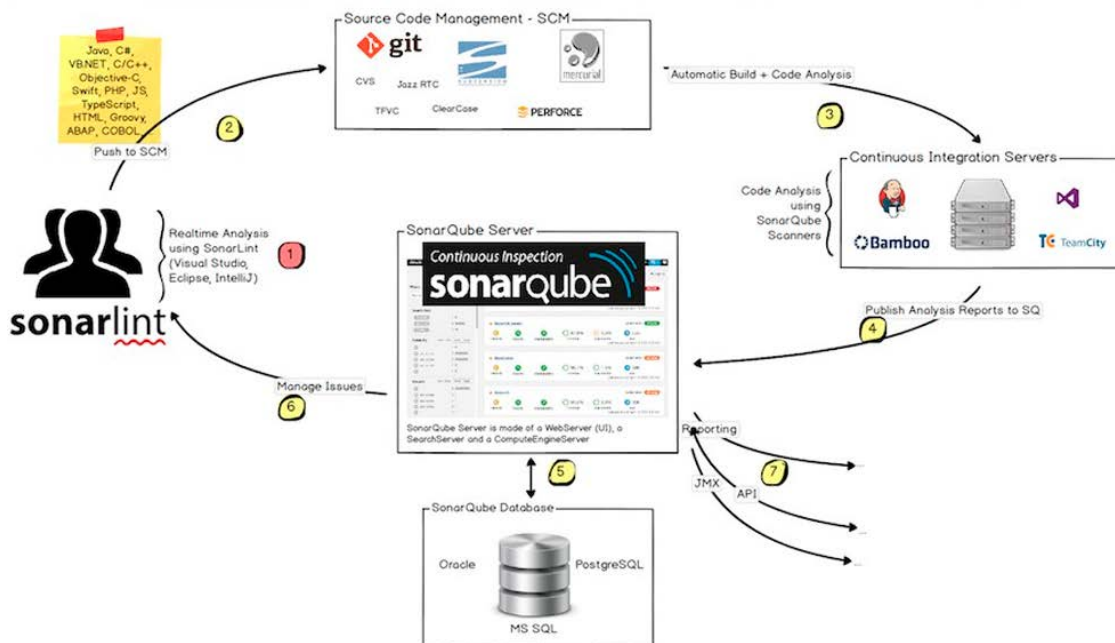
Obs.: O Sonar se integra ao Visual Studio, permitindo que, no momento da programação, seja possível visualizar os problemas existentes, como bugs de aplicação, débito técnico, *code smells* e security.

PLATAFORMA



Obs.: A plataforma funciona da seguinte forma: há a linguagem de programação, o escaneador do Sonar e um servidor. Existem algumas ferramentas envolvidas que exibem dashboards. As informações são armazenadas em um banco de dados, dependendo da versão utilizada. Normalmente, utiliza-se o PostgreSQL para esse armazenamento. O acesso às análises é feito por meio do servidor web, onde são visualizados os dashboards das aplicações enviadas. No caso específico, utilizava-se PLSQL, pois o Oracle Forms, por trás, possui muito PLSQL, e a plataforma também apresenta essas informações.

INTEGRAÇÃO



Obs.: Quanto à integração, existem algumas outras ferramentas, como o SonarLint, que se integra ao versionador, como o Git, ou outro sistema de controle de versão. Assim, antes de o código ser enviado para produção, ele passa pelo Sonar, que apresenta a situação do código antes de subir para produção.

MÉTRICAS

Complexidade: É a complexidade ciclomática calculada com base no número de caminhos através do código. Sempre que o fluxo de controle de uma função é dividido, o contador de complexidade é incrementado em um. Cada função tem uma complexidade mínima de 1. Esse cálculo varia um pouco por idioma, como fazem as palavras-chave e as funcionalidades.

Obs.: A complexidade ciclomática corresponde ao número de caminhos existentes no código-fonte, considerando estruturas como *ifs* e *loops*. Quanto maior o número de voltas, maior a complexidade ciclomática. O Sonar realiza a análise desse aspecto.

DUPLICAÇÕES:

- Blocos duplicados (*duplicated_blocks*) - Número de blocos duplicados de linhas.
- Arquivos duplicados (*duplicated_files*) - Número de arquivos envolvidos nas duplicações.
- Linhas duplicadas (*duplicated_lines*) - Número de linhas envolvidas nas duplicações.
- Linhas duplicadas (%) (*duplicated_lines_density*) = $\text{duplicated_lines} / \text{lines} * 100$

PROBLEMAS:

- Problemas de falsos positivos (*false_positive_issues*) Contagem total de problemas marcados como falsos positivos.
- Problemas em aberto (*open_issues*) - Contagem total de problemas no estado Aberto.
- Problemas confirmados (*confirmed_issues*) - Contagem total de problemas no estado Confirmado.
- Problemas reabertos (*reopened_issues*) - Contagem total de problemas no estado Reaberto.

Obs.: O Sonar permite integração para abertura de *issues* no código, contabilizando a quantidade de *issues* existentes.

MANUTENÇÃO:

- Esforço da dívida técnica (*sqale_index*) - para corrigir todos os odores de código. A medida é armazenada em minutos no banco de dados. Um dia de 8 horas é assumido quando os valores são mostrados em dias.

Obs.: Conforme observado na imagem do painel do SonarQube, há a informação de dois dias de débito. Isso significa que, ao identificar os problemas, o sistema calculou e armazenou o tempo estimado para a resolução, indicando que são dois dias de trabalho necessários para resolver os problemas apresentados.

- Dívida técnica no novo código (new_technical_debt) Esforço para corrigir todos os odores de código levantados pela primeira vez no período do novo código.

Obs.: A dívida técnica no novo código corresponde ao esforço necessário para corrigir os problemas identificados. Por exemplo, quando o código apresenta nomes de variáveis inadequados, desrespeitando o padrão de codificação, ou quando o código contém diversas gambiarras, o Sonar identifica essas falhas. Isso ocorre porque o Sonar aplica os conceitos de *clean code*, analisando o código e apontando as práticas que não estão sendo seguidas.

Quality Gates: Status do Quality Gate (alert_status) – Estado do Quality Gate associado ao seu projeto. Os valores possíveis são: ERROR, OKWARN valor foi removido desde 7.6.

Obs.: Quality Gates corresponde ao estado final, no qual há um percentual mínimo estabelecido. É muito comum, especialmente em órgãos governamentais, que editais determinem a exigência do Quality Gate. Nesses casos, toda vez que o fornecedor entrega um código, este deve apresentar cobertura de teste mínima de cem por cento. Embora essa exigência seja rigorosa, ela é frequentemente aplicada, incluindo a necessidade de testes unitários completos.

Obs.: Além disso, o débito técnico não pode exceder três dias, a quantidade de bugs deve estar dentro de um limite estabelecido, assim como a duplicação de linhas deve respeitar um determinado percentual. São definidas metas consideradas razoáveis. De acordo com a linguagem de programação utilizada, esses padrões podem ser adotados ou personalizados.

CONFIABILIDADE:

- Bugs(bugs) – Número de problemas de bugs.
- Novos erros(new_bugs) – Número de novos problemas de erros.
- Classificação de confiabilidade (reliability_rating):

A = 0 erros;

B = pelo menos 1 erro menor;

C = pelo menos 1 erro grave;

D = pelo menos 1 erro crítico;

E = pelo menos 1 erro bloqueador.

SEGURANÇA:

- Vulnerabilidades (vulnerabilities) – Número de problemas de vulnerabilidade.

- Novas vulnerabilidades(`new_vulnerabilities`) - Número de novos problemas de vulnerabilidade.

TAMANHO

- Classes (classes): Número de classes (incluindo classes aninhadas, interfaces, enumerações e anotações).
- Linhas de comentário (comment_lines) Número de linhas que contêm código de comentário ou comentário.

TESTES

- Cobertura da condição no novo código (new_branch_coverage) - Idêntica à cobertura da condição, mas restrita ao código fonte novo / atualizado.
- Resultados da cobertura da condição (branch_coverage_hits_data) - Lista de condições cobertas.
- Condições por linha (conditions_by_line) - Número de condições por linha.

REGRAS

- No SonarQube, os analisadores contribuem com regras executadas no código fonte para gerar problemas. Existem quatro tipos de regras:
 1. Cheiro de código (domínio de manutenção);
 2. Bug(domínio de confiabilidade);
 3. Vulnerabilidade (domínio de segurança);
 4. Ponto de acesso de segurança (domínio de segurança).

Obs.: Nas telas, a divisão ocorre da seguinte forma: a presença de *code smells* prejudica a manutenibilidade; quanto maior a quantidade de *bugs*, mais problemas a aplicação apresenta, afetando a confiabilidade; e as vulnerabilidades, que são pontos de acesso relacionados à segurança, comprometem a segurança da aplicação.

- Para odores e erros de código, são esperados zero falsos positivos. Pelo menos esse é o objetivo, para que os desenvolvedores não precisem se perguntar se é necessária uma correção.
- Para as vulnerabilidades, o objetivo é fazer com que mais de 80% dos problemas sejam verdadeiros positivos.

Obs.: Ou seja, quando houver um problema que ultrapasse 80%, ele será considerado um problema, mesmo que o sistema não o aponte erroneamente.

- As regras de ponto de acesso de segurança chamam a atenção para códigos sensíveis à segurança. Espera-se que mais de 80% dos problemas sejam resolvidos rapidamente como “Revisados” após a revisão por um desenvolvedor.
- A página Regras é o ponto de entrada onde você pode descobrir todas as regras existentes ou criar novas com base nos modelos fornecidos.

O Sonar não utiliza inteligência artificial, é muito mais algoritmo. Possui regras e, obviamente, ele vai detectando, sendo necessário afinar essas regras. No final, ninguém basicamente mexe nessas regras. Ele já tem uma precisão muito grande, então quando aponta, deve ser analisado. De vez em quando, pode-se ver algo que não seja um problema. Principalmente quando há contratos por trás e a empresa reclama que está muito rigoroso, verifica-se que o nível subiu. No dia a dia, para quem usa no mercado, o Sonar apenas aponta os problemas. Ao resolver os problemas, o código fica em nível adequado.



20m

CODE SMELLS

1. Um problema relacionado à manutenção no código.
2. Deixar como está significa que, na melhor das hipóteses, os mantenedores terão mais dificuldade do que deveriam fazer alterações no código.
3. Na pior das hipóteses, eles ficarão tão confusos com o estado do código que introduzirão erros adicionais à medida que fizerem alterações.

Exemplos:

- **Código duplicado:** código idêntico ou muito similar existe em mais de um local.
- **Método longo:** um método, função ou procedure muito extenso.
- **Classe extensa:** uma classe que acabou ficando muito extensa (GodObject).
- **Feature envy (sem tradução):** uma classe que utiliza em excesso métodos de outra classe.
- **Intimidade inapropriada:** uma classe que possui dependência de detalhes de implementação de outra classe.
- **Legado recusado:** uma classe que sobrepõe (override) o método da classe genérica de forma que o contrato da classe genérica não é cumprido pela classe derivada.
- **Classe preguiçosa:** classe que faz muito pouco.
- **Complexidade artificial:** uso forçado de design patterns extremamente complicados, onde um design simples seria suficiente.
- **Identificadores excessivamente longos:** em particular, o uso de convenções de nomes para evitar ambiguidades, o que deveria estar implícito na arquitetura do software.



DIRETO DO CONCURSO

01. (CESPE/TCE-PA/AUDITOR DE CONTROLE EXTERNO/ÁREA INFORMÁTICA/ANALISTA DE SISTEMA/2016) Acerca de análise estática de código-fonte, uma das práticas que verifica a qualidade do código e pode ser realizada antes da execução do software, julgue o próximo item.

A ferramenta SonarQube permite analisar a qualidade dos códigos-fontes que envolvem linguagens de computador e de dispositivos móveis e abrange categorias como padrões de codificação, testes e identificação de erros.



Justificativa: SonarQube é uma plataforma de código aberto desenvolvida pela SonarSource para inspeção contínua da qualidade do código para executar revisões automáticas com análise estática do código para detectar bugs, odores de código e vulnerabilidades de segurança em mais de 20 linguagens de programação.

02. (CESPE/STJ/TÉCNICO JUDICIÁRIO/DESENVOLVIMENTO DE SISTEMAS/2018) Julgue o item a seguir, acerca de eMAG, sistemas de controle de versão e SonarQube.

Uma issue gerada pelo SonarQube com severidade CRITICA requer a imediata correção do código.



Ele gera issues automaticamente ao identificar um problema, por isso existem indicadores de issues. O tratamento dessas issues depende do usuário. Conforme a classificação utilizada, se a issue for crítica, deve ser revisada; se for bloqueadora, deve ser corrigida. A correção imediata da issue bloqueadora é recomendada, mas não obrigatória. Algumas issues podem permanecer sem correção por longos períodos, dependendo do sistema. Quando o SonarQube indica uma issue crítica, recomenda-se revisão; para a issue bloqueadora, a correção é necessária. A decisão sobre a urgência da correção cabe ao usuário.

Justificativa:

Issue com severidade CRITICA deve ser **revisada**.

Issue com severidade BLOCKER deve ser **corrigida**.

BLOCKER - Bug with a high probability to impact the behavior of the application in production: memory leak, unclosed JDBC connection,.... The code **MUST** be immediately fixed.



25m

CRITICAL - Either a bug with a low probability to impact the behavior of the application in production or an issue which represents a security flaw: empty catch block, SQL injection,... The code **MUST** be immediately reviewed.

Fonte: <https://docs.sonarqube.org/display/SONAR/Issue>.

03. (CESPE/FUNPRES-P-EXE/ESPECIALISTA/TECNOLOGIA DA INFORMAÇÃO) A respeito da ferramenta SonarQube e da interoperabilidade de sistemas, julgue o seguinte item.

No SonarQube, mesmo com as permissões de segurança, nem todas as tags de regras podem ser removidas.



As tags de regra são configurações feitas na ferramenta para definir o que será analisado durante a execução da análise. Pode ocorrer o chamado falso positivo, que é quando a ferramenta aponta um problema que, na verdade, não é um problema. Assim como ocorre em sistemas de bloqueio, como o firewall, que às vezes bloqueia algo que não deveria, o sonar também pode indicar problemas inexistentes. Nem todas as tags de regra podem ser removidas, pois algumas são essenciais, como a verificação de duplicação de código. Caso se deseje ignorar completamente as análises, a ferramenta não deveria ser usada. Um exemplo comum de tentativa de burlar o sistema é a criação de testes unitários mínimos, que servem apenas para passar pela regra obrigatória, sem realmente testar adequadamente. Essa prática é problema do usuário que realiza a burla.

Justificativa:

As duas ações a seguir estão disponíveis apenas se o usuário tiver as permissões corretas ("Administrar perfis e portões de qualidade").

Adicionar / remover tags:

- * É possível adicionar tags existentes em uma regra ou criar novas (basta inserir um novo nome ao digitar no campo de texto).
- * Observe que algumas regras possuem tags internas que o usuário não pode remover – elas são fornecidas pelos plug-ins que contribuem com as regras.

Estender a descrição:

- * A extensão das descrições de regras é útil para informar aos usuários como sua organização está usando uma regra específica, por exemplo, ou para fornecer mais informações sobre uma regra. Observe que a extensão estará disponível para usuários não administradores como parte normal dos detalhes da regra.

Fonte: <http://docs.sonarqube.org/display/SONAR/Rules>.

04. (CESPE/CEBRASPE/MPE-GO ANALISTA EM INFORMÁTICA/2024) Julgue o item subsequente, relativo a WebServices, SonarQube.

O sonarQube detecta as vulnerabilidades de segurança no código por meio de DAST (dynamic application security testing), indicando os erros a serem eliminados antes de implantar o aplicativo.



O conceito de SAST e DAST está diretamente relacionado ao sonar. O sonar é uma ferramenta de SAST, ou seja, uma análise estática, não dinâmica. Isso significa que ele não realiza testes de segurança dinâmicos na aplicação, pois a aplicação não precisa ser executada para a análise. Esse é o conceito básico no teste de software.

Existem testes de defeito que procuram identificar falhas na aplicação por meio da execução da mesma, conhecidos como testes dinâmicos. O sonar, por sua vez, analisa o código-fonte estático e aponta diversos problemas, como nomes inadequados de variáveis e métodos, que podem gerar dificuldades no código.

De acordo com o livro do Presmo, cerca de 60% dos problemas em código podem ser detectados apenas com análise estática. Esse dado, apesar de antigo, ainda é considerado fundamental. Em alguns locais, o uso do sonar melhorou significativamente a qualidade do código, principalmente quando os testes unitários não eram realizados adequadamente.

As análises estática e dinâmica são técnicas complementares. A análise estática não substitui testes dinâmicos, como testes unitários, de integração, de sistema e de aceitação, pois uma aplicação pode passar no sonar e ainda assim não atender aos requisitos funcionais ou satisfazer o usuário. Portanto, essas estratégias são complementares.

Justificativa:

DAST – dynamic application security testing.

SAST –static application security testing.

05. (FGV/TRF-1ª REGIÃO/ANALISTA JUDICIÁRIO/APOIO ESPECIALIZADO/ANÁLISE DE SISTEMAS DE INFORMAÇÃO/2024) O analista Jeferson está configurando um projeto no SonarQube. Ao configurar as métricas de código que devem ser monitoradas, Jefferson adicionou ao projeto a métrica que indica o tempo necessário para corrigir todos os code smells presentes no código. Jefferson adicionou ao projeto no SonarQube a métrica:

a) technical debt;

- b) quality gate status;
- c) security remediation;
- d) maintainability rating;
- e) reliability remediation.



Débito técnico é aquele que corresponde aos 2 dias de débito técnico.

06. (CEBRASPE/PC-DF/GESTOR DE APOIO AS ATIVIDADES POLICIAIS CIVIS/ANALISTA DE INFORMÁTICA/DESENVOLVIMENTO DE SISTEMAS/2025) Julgue o item que se segue, referente a clean code e à ferramenta SonarQube.

O SonarQube tem uma abordagem embasada no princípio de que o código da base em produção deve ser formatado e revisado para garantir que se usem as interfaces de programação e os recursos de linguagem apropriados.



Sonarqube é uma ferramenta para análise estática. Ela não serve apenas para verificar interface, programação ou recurso de linguagem.

Justificativa: O SonarQube tem uma abordagem baseada no princípio de que o novo código (código adicionado ou modificado recentemente pelo usuário) deve estar em conformidade com os padrões de qualidade do SonarQube (servidor, nuvem). A solução Sonar implementa o clean as you code, avisando sempre que problemas forem detectados no novo código.

07. (CESPE/CEBRASPE/DATAPREV/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO/DESENVOLVIMENTO DE SOFTWARE/2023) Julgue o item abaixo, relacionados com JavaScript, Web Services e análise estatística de código fonte.

No SonarQube, a complexidade mede a quantidade de caminhos possíveis na execução do código.



Justificativa:

Complexidade ciclomática mede a quantidade de caminhos possíveis na execução do código.

GABARITO

01. C

02. E

03. C

04. E

05. a

06. E

07. c

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pelo professor Washington Henrique Almeida.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
